APOSTILA – ESTRUTURAS DE REPETIÇÃO E MÉTODOS (C#)

Introdução

Durante o desenvolvimento de programas, frequentemente precisamos executar uma mesma ação várias vezes ou organizar nosso código em partes reutilizáveis. Para isso, utilizamos as **estruturas de repetição** e os **métodos**.

Nesta apostila você aprenderá:

- Estrutura `for`
- Estrutura `while`
- Estrutura `foreach`
- Métodos
- Parâmetros
- Exemplos práticos
- Exercícios de fixação

---

1. Estruturas de Repetição

As estruturas de repetição permitem executar um bloco de código diversas vezes sem precisar escrevê-lo repetidamente.

As principais estruturas são:

- `for`
- `while`
- `foreach`

---

2. Estrutura FOR

A estrutura `for` é utilizada quando sabemos previamente quantas vezes o laço será executado.

Sintaxe

```
for (inicialização; condição; incremento)
{
   // código a ser repetido
}
```

Exemplo

```
for (int i = 1; i <= 5; i++)
{
   Console.WriteLine(i);
}
```

Saída

```
1
2
3
4
5
```

Explicação

- `int i = 1` → valor inicial.
- `i <= 5` → condição para continuar repetindo.
- `i++` → incrementa 1 a cada repetição.

Exemplo: Somando números

```
int soma = 0;

for (int i = 1; i <= 10; i++)
{
   soma += i;
}

Console.WriteLine(soma);
```

Resultado:

```
55
```

---

3. Estrutura WHILE

A estrutura `while` executa um bloco enquanto uma condição for verdadeira.

Sintaxe

```
while (condição)
{
  // código
}
```

Exemplo

```
int contador = 1;

while (contador <= 5)
{
  Console.WriteLine(contador);
  contador++;
}
```

Saída

```
1
2
3
4
5
```

Exemplo: Validando entrada

```
int numero = 0;

while (numero <= 0)
{
  Console.Write("Digite um número positivo: ");
  numero = int.Parse(Console.ReadLine());
}
```

O laço continuará executando até que o usuário informe um número maior que zero.

---

4. Diferença entre FOR e WHILE

FOR	WHILE

Usado quando sabemos a quantidade de repetições	Usado quando não sabemos quantas repetições serão necessárias
Controle da variável fica na própria estrutura	Controle normalmente é feito dentro do bloco
Mais compacto	Mais flexível

---

5. Estrutura FOREACH

O `foreach` é utilizado para percorrer coleções como arrays e listas.

Sintaxe

```
foreach (tipo variavel in coleção)
{
  // código
}
```

Exemplo com Array

```
string[] nomes =
{
  "Ana",
  "Carlos",
  "Maria"
};

foreach (string nome in nomes)
{
  Console.WriteLine(nome);
}
```

Saída

```
Ana
Carlos
Maria
```

Exemplo com Lista

```
List<int> numeros = new List<int>()
{
```

```
  10,
  20,
  30,
  40
};

foreach (int numero in numeros)
{
  Console.WriteLine(numero);
}
```

Saída

```
10
20
30
40
```

---

6. Quando Utilizar FOREACH

Utilize `foreach` quando:

- Não precisar do índice da posição.
- Apenas quiser percorrer os elementos.
- Desejar um código mais simples e legível.

Exemplo:

```
foreach (var produto in produtos)
{
  Console.WriteLine(produto.Nome);
}
```

---

7. Métodos

Métodos são blocos de código reutilizáveis que executam uma tarefa específica.

Vantagens

- Organização do código.
- Reutilização.
- Facilidade de manutenção.
- Melhor legibilidade.

Estrutura de um Método

```
public static void NomeMetodo()
{
   // código
}
```

Exemplo

```
public static void Saudacao()
{
   Console.WriteLine("Bem-vindo ao sistema!");
}
```

Chamando o método:

```
Saudacao();
```

Resultado:

```
Bem-vindo ao sistema!
```

---

8. Métodos com Retorno

Métodos podem retornar um valor.

Exemplo

```
public static int Somar()
{
   return 10 + 20;
}
```

Chamando:

```
int resultado = Somar();

Console.WriteLine(resultado);
```

Resultado:

```
30
```

---

9. Parâmetros

Parâmetros são informações enviadas para um método.

Exemplo

```
public static void MostrarNome(string nome)
{
  Console.WriteLine(nome);
}
```

Chamada:

```
MostrarNome("João");
```

Resultado:

João

---

10. Métodos com Vários Parâmetros

```
public static void Somar(int numero1, int numero2)
{
  Console.WriteLine(numero1 + numero2);
}
```

Chamada:

```
Somar(10, 5);
```

Resultado:

15

---

11. Método com Retorno e Parâmetros

```
public static int Multiplicar(int a, int b)
{
  return a * b;
}
```

Chamada:

```
int resultado = Multiplicar(4, 5);
```

Console.WriteLine(resultado);

Resultado:

20

---

12. Exemplo Completo

```
using System;

class Program
{
  static void Main()
  {
     int resultado = Somar(10, 20);

     Console.WriteLine(resultado);
  }

  static int Somar(int a, int b)
  {
     return a + b;
  }
}
```

Saída:

30

Resumo

FOR

Utilizado quando sabemos quantas vezes o código será executado.

```
for(int i = 0; i < 10; i++)
{
}
```

WHILE

Executa enquanto a condição for verdadeira.

```
while(condicao)
{
}
```

FOREACH

Percorre coleções.

```
foreach(var item in lista)
{
}
```

MÉTODOS

Agrupam instruções reutilizáveis.

```
static void MeuMetodo()
{
}
```

PARÂMETROS

Permitem enviar dados ao método.

```
static void Mostrar(string nome)
{
}
```

EXERCÍCIOS

1

Faça um programa utilizando `for` que exiba os números de 1 a 20.

2

Faça um programa utilizando `for` que exiba apenas os números pares de 0 a 50.

3

Crie um programa que utilize `for` para calcular a soma dos números de 1 até 100.

4

Utilize `while` para exibir os números de 10 até 1 em ordem decrescente.

5

Faça um programa que solicite uma senha ao usuário e continue pedindo enquanto a senha digitada for diferente de `"1234"`.

6

Crie um array contendo 5 nomes e utilize `foreach` para exibi-los na tela.

7

Crie uma lista contendo 10 números inteiros e utilize `foreach` para calcular a soma de todos os elementos.

8

Crie um método chamado `Mensagem()` que exiba a frase:

Olá, mundo!

Depois chame o método no `Main`.

9

Crie um método chamado `Dobro(int numero)` que receba um número por parâmetro e exiba seu dobro.

Exemplo:

Entrada: 5
Saída: 10

10

Crie um método chamado `Media(double nota1, double nota2)` que retorne a média das duas notas.

Exemplo:

Entrada: 8 e 6
Saída: 7